HW11-ICA-PartA_Adam_Nelson-Archer

April 16, 2026

1 HW11 Part A :: Image Classification using Vision Transformer

COSC 6373 – Adam Nelson-Archer, 2140122

1.1 Setup

[15]:
```
import os
import time

os.environ["KERAS_BACKEND"] = "tensorflow"

import keras
from keras import layers
from keras import ops

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

print("Keras:", keras.__version__)
print("TensorFlow:", tf.__version__)
```

```
Keras: 3.1.1
TensorFlow: 2.16.1
```

1.2 1. Prepare the dataset

1.2.1 a. Load the Horses vs. Camels dataset

1.2.2 b. Split the dataset into training, validation, and test sets (70/15/15)

1.2.3 c. Preprocessing: Resize to 224x224, normalize pixel values

[16]:
```
import kagglehub
from pathlib import Path

download_root = Path(kagglehub.dataset_download("akrsnv/horses-and-camels"))
print("Downloaded to:", download_root)

# Inspect the dataset directory
```

```python
for p in sorted(download_root.rglob("*"))[:20]:
    print(p)
```

```
Downloaded to: C:\Users\Adam\.cache\kagglehub\datasets\akrsnv\horses-and-
camels\versions\2
C:\Users\Adam\.cache\kagglehub\datasets\akrsnv\horses-and-camels\versions\2\test
C:\Users\Adam\.cache\kagglehub\datasets\akrsnv\horses-and-
camels\versions\2\test\camel
C:\Users\Adam\.cache\kagglehub\datasets\akrsnv\horses-and-
camels\versions\2\test\camel\180.png
C:\Users\Adam\.cache\kagglehub\datasets\akrsnv\horses-and-
camels\versions\2\test\camel\181.png
C:\Users\Adam\.cache\kagglehub\datasets\akrsnv\horses-and-
camels\versions\2\test\camel\182.png
C:\Users\Adam\.cache\kagglehub\datasets\akrsnv\horses-and-
camels\versions\2\test\camel\183.png
C:\Users\Adam\.cache\kagglehub\datasets\akrsnv\horses-and-
camels\versions\2\test\camel\184.png
C:\Users\Adam\.cache\kagglehub\datasets\akrsnv\horses-and-
camels\versions\2\test\camel\185.png
C:\Users\Adam\.cache\kagglehub\datasets\akrsnv\horses-and-
camels\versions\2\test\camel\186.png
C:\Users\Adam\.cache\kagglehub\datasets\akrsnv\horses-and-
camels\versions\2\test\camel\187.png
C:\Users\Adam\.cache\kagglehub\datasets\akrsnv\horses-and-
camels\versions\2\test\camel\188.png
C:\Users\Adam\.cache\kagglehub\datasets\akrsnv\horses-and-
camels\versions\2\test\camel\189.png
C:\Users\Adam\.cache\kagglehub\datasets\akrsnv\horses-and-
camels\versions\2\test\camel\190.png
C:\Users\Adam\.cache\kagglehub\datasets\akrsnv\horses-and-
camels\versions\2\test\camel\191.png
C:\Users\Adam\.cache\kagglehub\datasets\akrsnv\horses-and-
camels\versions\2\test\camel\192.png
C:\Users\Adam\.cache\kagglehub\datasets\akrsnv\horses-and-
camels\versions\2\test\camel\193.png
C:\Users\Adam\.cache\kagglehub\datasets\akrsnv\horses-and-
camels\versions\2\test\camel\194.png
C:\Users\Adam\.cache\kagglehub\datasets\akrsnv\horses-and-
camels\versions\2\test\camel\195.png
C:\Users\Adam\.cache\kagglehub\datasets\akrsnv\horses-and-
camels\versions\2\test\camel\196.png
C:\Users\Adam\.cache\kagglehub\datasets\akrsnv\horses-and-
camels\versions\2\test\camel\197.png
```

[17]:
```python
image_size = 224
batch_size = 32
```

```
# The dataset has train/ and test/ folders, each with camel/ and horse/␣
↪subfolders.
# We merge everything into one pool and do our own 70/15/15 split.
import glob
from tensorflow.keras.preprocessing.image import load_img, img_to_array

all_images = []
all_labels = []
class_names = ["camel", "horse"]

for split_folder in ["train", "test"]:
    for label_idx, cls_name in enumerate(class_names):
        folder = download_root / split_folder / cls_name
        if not folder.exists():
            continue
        for img_path in sorted(folder.glob("*.png")):
            img = load_img(img_path, target_size=(image_size, image_size))
            all_images.append(img_to_array(img))
            all_labels.append(label_idx)

images = np.array(all_images, dtype="float32") / 255.0
labels_arr = np.array(all_labels)

num_classes = len(class_names)
print(f"Classes: {class_names}")
print(f"Total images: {len(images)}, shape: {images.shape}")
print(f"  Camels: {np.sum(labels_arr == 0)}, Horses: {np.sum(labels_arr == 1)}")
```

```
Classes: ['camel', 'horse']
Total images: 400, shape: (400, 224, 224, 3)
  Camels: 200, Horses: 200
```

[18]:
```
# Split: 70% train, 15% val, 15% test
from sklearn.model_selection import train_test_split

x_temp, x_test, y_temp, y_test = train_test_split(
    images, labels_arr, test_size=0.15, random_state=42, stratify=labels_arr
)

# Split the remaining 85% into train (70/85 ~ 82.35%) and val (15/85 ~ 17.65%)
x_train, x_val, y_train, y_val = train_test_split(
    x_temp, y_temp, test_size=15/85, random_state=42, stratify=y_temp
)

print(f"Train: {x_train.shape[0]} images")
print(f"Val:   {x_val.shape[0]} images")
```

```
print(f"Test:  {x_test.shape[0]} images")
```

```
Train: 279 images
Val:   61 images
Test:  60 images
```

[19]:
```
# Visualize a few samples
plt.figure(figsize=(10, 4))
for i in range(8):
    plt.subplot(2, 4, i + 1)
    plt.imshow(x_train[i])
    plt.title(class_names[y_train[i]])
    plt.axis("off")
plt.suptitle("Sample Training Images")
plt.tight_layout()
plt.show()
```

1.3 2. Build the ViT model

Using the provided Keras starter code. No pretrained weights – model is initialized from scratch.

1.3.1 Configure hyperparameters

These training settings will be reused for the CNN in Part B to ensure a fair comparison.

[20]:
```
# --- Training settings (shared with CNN in Part B) ---
learning_rate = 0.001
weight_decay = 0.0001
num_epochs = 10

# --- ViT-specific hyperparameters ---
```

```
patch_size = 16
num_patches = (image_size // patch_size) ** 2  # 196 patches for 224x224 with␣
↪patch 16
projection_dim = 64
num_heads = 4
transformer_units = [
    projection_dim * 2,
    projection_dim,
]
transformer_layers = 8
mlp_head_units = [
    2048,
    1024,
]

input_shape = (image_size, image_size, 3)

print(f"Image size: {image_size}x{image_size}")
print(f"Patch size: {patch_size}x{patch_size}")
print(f"Number of patches: {num_patches}")
print(f"Projection dim: {projection_dim}")
```

```
Image size: 224x224
Patch size: 16x16
Number of patches: 196
Projection dim: 64
```

1.3.2 Data augmentation

[21]:
```
data_augmentation = keras.Sequential(
    [
        layers.Normalization(),
        layers.RandomFlip("horizontal"),
        layers.RandomRotation(factor=0.02),
        layers.RandomZoom(height_factor=0.2, width_factor=0.2),
    ],
    name="data_augmentation",
)

# Compute the mean and the variance of the training data for normalization.
data_augmentation.layers[0].adapt(x_train)
```

1.3.3 Implement multilayer perceptron (MLP)

[22]:
```
def mlp(x, hidden_units, dropout_rate):
    for units in hidden_units:
        x = layers.Dense(units, activation=keras.activations.gelu)(x)
```

```
        x = layers.Dropout(dropout_rate)(x)
    return x
```

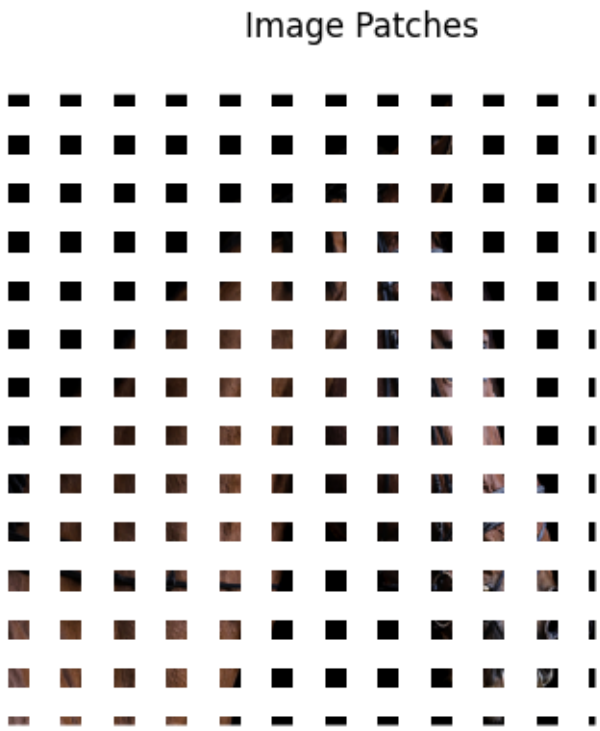
1.3.4 Implement patch creation as a layer

[23]:
```
class Patches(layers.Layer):
    def __init__(self, patch_size):
        super().__init__()
        self.patch_size = patch_size

    def call(self, images):
        input_shape = ops.shape(images)
        batch_size = input_shape[0]
        height = input_shape[1]
        width = input_shape[2]
        channels = input_shape[3]
        num_patches_h = height // self.patch_size
        num_patches_w = width // self.patch_size
        patches = keras.ops.image.extract_patches(images, size=self.patch_size)
        patches = ops.reshape(
            patches,
            (
                batch_size,
                num_patches_h * num_patches_w,
                self.patch_size * self.patch_size * channels,
            ),
        )
        return patches

    def get_config(self):
        config = super().get_config()
        config.update({"patch_size": self.patch_size})
        return config
```

[24]:
```
# Display patches for a sample image
plt.figure(figsize=(4, 4))
image = x_train[np.random.choice(range(x_train.shape[0]))]
plt.imshow(image)
plt.axis("off")

resized_image = ops.image.resize(
    ops.convert_to_tensor([image]), size=(image_size, image_size)
)
patches = Patches(patch_size)(resized_image)
print(f"Image size: {image_size} X {image_size}")
print(f"Patch size: {patch_size} X {patch_size}")
print(f"Patches per image: {patches.shape[1]}")
```

```python
print(f"Elements per patch: {patches.shape[-1]}")

n = int(np.sqrt(patches.shape[1]))
plt.figure(figsize=(4, 4))
for i, patch in enumerate(patches[0]):
    ax = plt.subplot(n, n, i + 1)
    patch_img = ops.reshape(patch, (patch_size, patch_size, 3))
    plt.imshow(ops.convert_to_numpy(patch_img))
    plt.axis("off")
plt.suptitle("Image Patches")
plt.tight_layout()
plt.show()
```

```
Image size: 224 X 224
Patch size: 16 X 16
Patches per image: 196
Elements per patch: 768
```



1.3.5 Implement the patch encoding layer

[25]:
```
class PatchEncoder(layers.Layer):
    def __init__(self, num_patches, projection_dim):
        super().__init__()
        self.num_patches = num_patches
        self.projection = layers.Dense(units=projection_dim)
        self.position_embedding = layers.Embedding(
            input_dim=num_patches, output_dim=projection_dim
        )

    def call(self, patch):
        positions = ops.expand_dims(
            ops.arange(start=0, stop=self.num_patches, step=1), axis=0
        )
        projected_patches = self.projection(patch)
        encoded = projected_patches + self.position_embedding(positions)
        return encoded

    def get_config(self):
        config = super().get_config()
        config.update({"num_patches": self.num_patches})
        return config
```

1.3.6 Build the ViT model

The model includes: - Patch extraction - Patch embedding - Transformer encoder layers - Classification head

[26]:
```
def create_vit_classifier():
    inputs = keras.Input(shape=input_shape)
    # Augment data.
    augmented = data_augmentation(inputs)
    # Create patches.
    patches = Patches(patch_size)(augmented)
    # Encode patches.
    encoded_patches = PatchEncoder(num_patches, projection_dim)(patches)

    # Create multiple layers of the Transformer block.
    for _ in range(transformer_layers):
        # Layer normalization 1.
        x1 = layers.LayerNormalization(epsilon=1e-6)(encoded_patches)
        # Create a multi-head attention layer.
        attention_output = layers.MultiHeadAttention(
            num_heads=num_heads, key_dim=projection_dim, dropout=0.1
        )(x1, x1)
        # Skip connection 1.
        x2 = layers.Add()([attention_output, encoded_patches])
        # Layer normalization 2.
```

```
        x3 = layers.LayerNormalization(epsilon=1e-6)(x2)
        # MLP.
        x3 = mlp(x3, hidden_units=transformer_units, dropout_rate=0.1)
        # Skip connection 2.
        encoded_patches = layers.Add()([x3, x2])

    # Create a [batch_size, projection_dim] tensor.
    representation = layers.LayerNormalization(epsilon=1e-6)(encoded_patches)
    representation = layers.Flatten()(representation)
    representation = layers.Dropout(0.5)(representation)
    # Add MLP.
    features = mlp(representation, hidden_units=mlp_head_units, dropout_rate=0.
↪5)
    # Classify outputs.
    logits = layers.Dense(num_classes)(features)
    # Create the Keras model.
    model = keras.Model(inputs=inputs, outputs=logits)
    return model

vit_classifier = create_vit_classifier()
vit_classifier.summary()
```

Model: "functional_5"

Layer (type)	Output Shape	Param #	Connected to
input_layer_2 (InputLayer)	(None, 224, 224, 3)	0	-
data_augmentation (Sequential)	(None, 224, 224, 3)	7	input_layer_2[0]…
patches_3 (Patches)	(None, 196, 768)	0	data_augmentatio…
patch_encoder_1 (PatchEncoder)	(None, 196, 64)	61,760	patches_3[0][0]
layer_normalizatio… (LayerNormalizatio…	(None, 196, 64)	128	patch_encoder_1[…
multi_head_attenti… (MultiHeadAttentio…	(None, 196, 64)	66,368	layer_normalizat… layer_normalizat…
add_16 (Add)	(None, 196, 64)	0	multi_head_atten… patch_encoder_1[…

layer_normalizatio… (LayerNormalizatio…	(None, 196, 64)	128	add_16[0][0]
dense_21 (Dense)	(None, 196, 128)	8,320	layer_normalizat…
dropout_28 (Dropout)	(None, 196, 128)	0	dense_21[0][0]
dense_22 (Dense)	(None, 196, 64)	8,256	dropout_28[0][0]
dropout_29 (Dropout)	(None, 196, 64)	0	dense_22[0][0]
add_17 (Add)	(None, 196, 64)	0	dropout_29[0][0], add_16[0][0]
layer_normalizatio… (LayerNormalizatio…	(None, 196, 64)	128	add_17[0][0]
multi_head_attenti… (MultiHeadAttentio…	(None, 196, 64)	66,368	layer_normalizat… layer_normalizat…
add_18 (Add)	(None, 196, 64)	0	multi_head_atten… add_17[0][0]
layer_normalizatio… (LayerNormalizatio…	(None, 196, 64)	128	add_18[0][0]
dense_23 (Dense)	(None, 196, 128)	8,320	layer_normalizat…
dropout_31 (Dropout)	(None, 196, 128)	0	dense_23[0][0]
dense_24 (Dense)	(None, 196, 64)	8,256	dropout_31[0][0]
dropout_32 (Dropout)	(None, 196, 64)	0	dense_24[0][0]
add_19 (Add)	(None, 196, 64)	0	dropout_32[0][0], add_18[0][0]
layer_normalizatio… (LayerNormalizatio…	(None, 196, 64)	128	add_19[0][0]
multi_head_attenti… (MultiHeadAttentio…	(None, 196, 64)	66,368	layer_normalizat… layer_normalizat…
add_20 (Add)	(None, 196, 64)	0	multi_head_atten…

			add_19[0][0]
layer_normalizatio… (LayerNormalizatio…	(None, 196, 64)	128	add_20[0][0]
dense_25 (Dense)	(None, 196, 128)	8,320	layer_normalizat…
dropout_34 (Dropout)	(None, 196, 128)	0	dense_25[0][0]
dense_26 (Dense)	(None, 196, 64)	8,256	dropout_34[0][0]
dropout_35 (Dropout)	(None, 196, 64)	0	dense_26[0][0]
add_21 (Add)	(None, 196, 64)	0	dropout_35[0][0], add_20[0][0]
layer_normalizatio… (LayerNormalizatio…	(None, 196, 64)	128	add_21[0][0]
multi_head_attenti… (MultiHeadAttentio…	(None, 196, 64)	66,368	layer_normalizat… layer_normalizat…
add_22 (Add)	(None, 196, 64)	0	multi_head_atten… add_21[0][0]
layer_normalizatio… (LayerNormalizatio…	(None, 196, 64)	128	add_22[0][0]
dense_27 (Dense)	(None, 196, 128)	8,320	layer_normalizat…
dropout_37 (Dropout)	(None, 196, 128)	0	dense_27[0][0]
dense_28 (Dense)	(None, 196, 64)	8,256	dropout_37[0][0]
dropout_38 (Dropout)	(None, 196, 64)	0	dense_28[0][0]
add_23 (Add)	(None, 196, 64)	0	dropout_38[0][0], add_22[0][0]
layer_normalizatio… (LayerNormalizatio…	(None, 196, 64)	128	add_23[0][0]
multi_head_attenti… (MultiHeadAttentio…	(None, 196, 64)	66,368	layer_normalizat… layer_normalizat…

add_24 (Add)	(None, 196, 64)	0	multi_head_atten… add_23[0][0]
layer_normalizatio… (LayerNormalizatio…	(None, 196, 64)	128	add_24[0][0]
dense_29 (Dense)	(None, 196, 128)	8,320	layer_normalizat…
dropout_40 (Dropout)	(None, 196, 128)	0	dense_29[0][0]
dense_30 (Dense)	(None, 196, 64)	8,256	dropout_40[0][0]
dropout_41 (Dropout)	(None, 196, 64)	0	dense_30[0][0]
add_25 (Add)	(None, 196, 64)	0	dropout_41[0][0], add_24[0][0]
layer_normalizatio… (LayerNormalizatio…	(None, 196, 64)	128	add_25[0][0]
multi_head_attenti… (MultiHeadAttentio…	(None, 196, 64)	66,368	layer_normalizat… layer_normalizat…
add_26 (Add)	(None, 196, 64)	0	multi_head_atten… add_25[0][0]
layer_normalizatio… (LayerNormalizatio…	(None, 196, 64)	128	add_26[0][0]
dense_31 (Dense)	(None, 196, 128)	8,320	layer_normalizat…
dropout_43 (Dropout)	(None, 196, 128)	0	dense_31[0][0]
dense_32 (Dense)	(None, 196, 64)	8,256	dropout_43[0][0]
dropout_44 (Dropout)	(None, 196, 64)	0	dense_32[0][0]
add_27 (Add)	(None, 196, 64)	0	dropout_44[0][0], add_26[0][0]
layer_normalizatio… (LayerNormalizatio…	(None, 196, 64)	128	add_27[0][0]

multi_head_attenti… (MultiHeadAttentio…	(None, 196, 64)	66,368	layer_normalizat… layer_normalizat…
add_28 (Add)	(None, 196, 64)	0	multi_head_atten… add_27[0][0]
layer_normalizatio… (LayerNormalizatio…	(None, 196, 64)	128	add_28[0][0]
dense_33 (Dense)	(None, 196, 128)	8,320	layer_normalizat…
dropout_46 (Dropout)	(None, 196, 128)	0	dense_33[0][0]
dense_34 (Dense)	(None, 196, 64)	8,256	dropout_46[0][0]
dropout_47 (Dropout)	(None, 196, 64)	0	dense_34[0][0]
add_29 (Add)	(None, 196, 64)	0	dropout_47[0][0], add_28[0][0]
layer_normalizatio… (LayerNormalizatio…	(None, 196, 64)	128	add_29[0][0]
multi_head_attenti… (MultiHeadAttentio…	(None, 196, 64)	66,368	layer_normalizat… layer_normalizat…
add_30 (Add)	(None, 196, 64)	0	multi_head_atten… add_29[0][0]
layer_normalizatio… (LayerNormalizatio…	(None, 196, 64)	128	add_30[0][0]
dense_35 (Dense)	(None, 196, 128)	8,320	layer_normalizat…
dropout_49 (Dropout)	(None, 196, 128)	0	dense_35[0][0]
dense_36 (Dense)	(None, 196, 64)	8,256	dropout_49[0][0]
dropout_50 (Dropout)	(None, 196, 64)	0	dense_36[0][0]
add_31 (Add)	(None, 196, 64)	0	dropout_50[0][0], add_30[0][0]
layer_normalizatio…	(None, 196, 64)	128	add_31[0][0]

```
(LayerNormalizatio…

flatten_1 (Flatten)   (None, 12544)                0   layer_normalizat…

dropout_51            (None, 12544)                0   flatten_1[0][0]
(Dropout)

dense_37 (Dense)      (None, 2048)        25,692,160   dropout_51[0][0]

dropout_52            (None, 2048)                 0   dense_37[0][0]
(Dropout)

dense_38 (Dense)      (None, 1024)         2,098,176   dropout_52[0][0]

dropout_53            (None, 1024)                 0   dense_38[0][0]
(Dropout)

dense_39 (Dense)      (None, 2)                2,050   dropout_53[0][0]


Total params: 28,519,881 (108.79 MB)


Trainable params: 28,519,874 (108.79 MB)


Non-trainable params: 7 (32.00 B)
```

1.4 3. Train the model

1.4.1 4. Training settings (shared with CNN in Part B)

- Batch size: 32
- Epochs: 10 (reduced from 20 so it could run in time, on laptop in class)
- Optimizer: AdamW
- Learning rate: 0.001

1.4.2 5. Record training time

[27]:
```
optimizer = keras.optimizers.AdamW(
    learning_rate=learning_rate, weight_decay=weight_decay
)

vit_classifier.compile(
    optimizer=optimizer,
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=[
        keras.metrics.SparseCategoricalAccuracy(name="accuracy"),
```

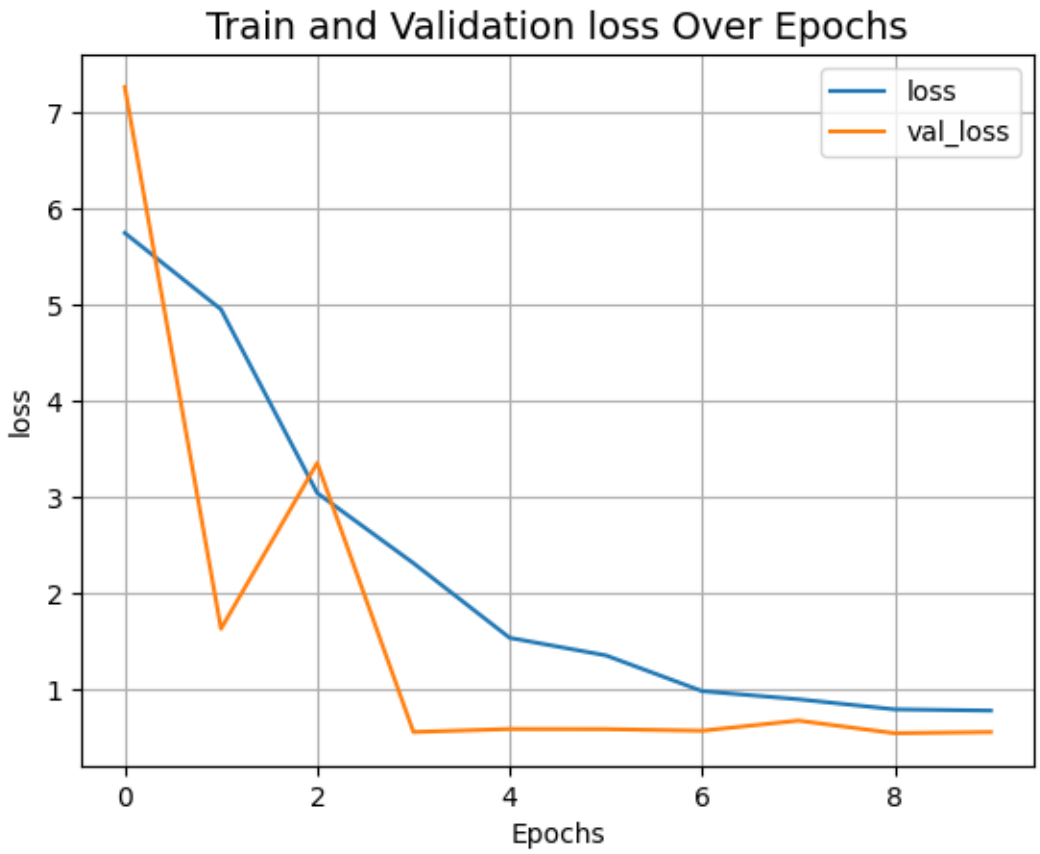
```
    ],
)

checkpoint_filepath = "vit_checkpoint.weights.h5"
checkpoint_callback = keras.callbacks.ModelCheckpoint(
    checkpoint_filepath,
    monitor="val_accuracy",
    save_best_only=True,
    save_weights_only=True,
)

# Record start time
start_time = time.time()

history = vit_classifier.fit(
    x=x_train,
    y=y_train,
    batch_size=batch_size,
    epochs=num_epochs,
    validation_data=(x_val, y_val),
    callbacks=[checkpoint_callback],
)

# Record end time
end_time = time.time()
training_duration = end_time - start_time

print(f"\nTraining start time: {time.strftime('%H:%M:%S', time.
 ↪localtime(start_time))}")
print(f"Training end time:   {time.strftime('%H:%M:%S', time.
 ↪localtime(end_time))}")
print(f"Total training duration: {training_duration:.2f} seconds␣
 ↪({training_duration/60:.2f} minutes)")
```

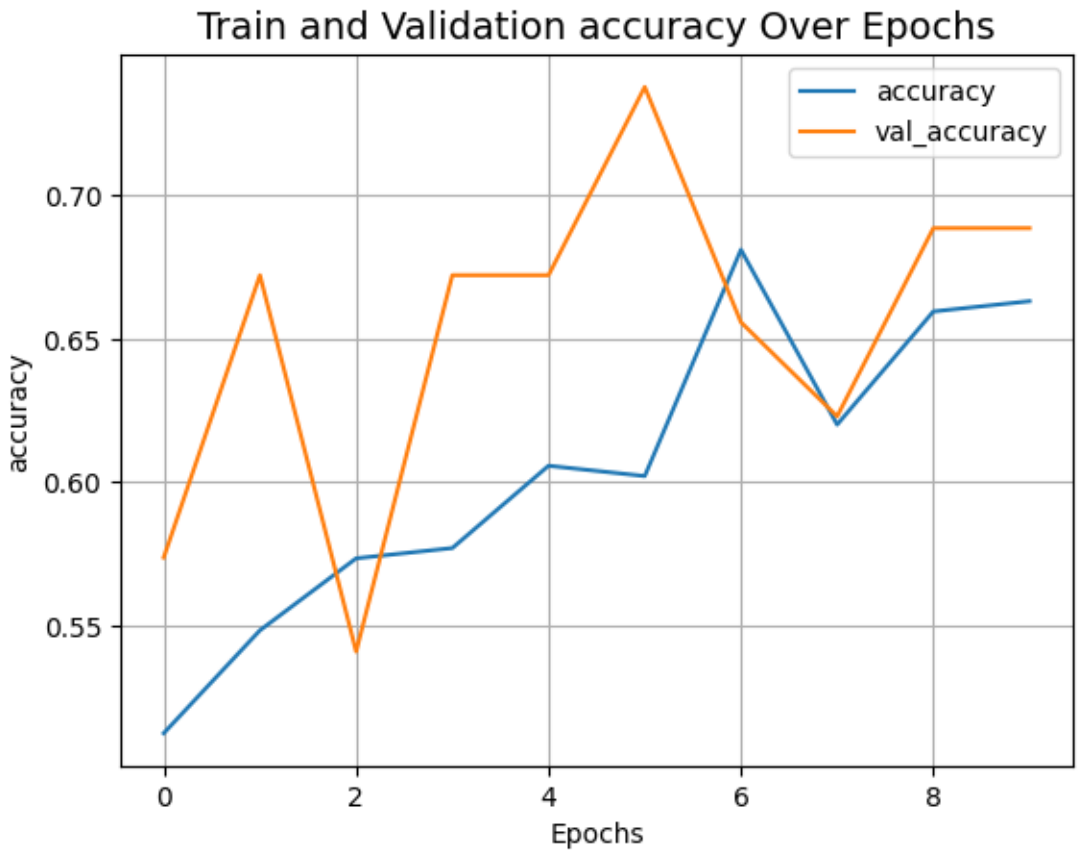
```
Epoch 1/10
9/9              60s 4s/step -
accuracy: 0.4797 - loss: 5.7854 - val_accuracy: 0.5738 - val_loss: 7.2613
Epoch 2/10
9/9              19s 2s/step -
accuracy: 0.5220 - loss: 5.6591 - val_accuracy: 0.6721 - val_loss: 1.6317
Epoch 3/10
9/9              18s 2s/step -
accuracy: 0.5600 - loss: 3.4082 - val_accuracy: 0.5410 - val_loss: 3.3548
Epoch 4/10
9/9              13s 1s/step -
accuracy: 0.5359 - loss: 3.1352 - val_accuracy: 0.6721 - val_loss: 0.5592
Epoch 5/10
```

```
9/9              13s 2s/step -
accuracy: 0.6110 - loss: 1.3951 - val_accuracy: 0.6721 - val_loss: 0.5902
Epoch 6/10
9/9              16s 2s/step -
accuracy: 0.6231 - loss: 1.2415 - val_accuracy: 0.7377 - val_loss: 0.5890
Epoch 7/10
9/9              12s 1s/step -
accuracy: 0.6851 - loss: 0.9583 - val_accuracy: 0.6557 - val_loss: 0.5724
Epoch 8/10
9/9              13s 1s/step -
accuracy: 0.6482 - loss: 0.9566 - val_accuracy: 0.6230 - val_loss: 0.6775
Epoch 9/10
9/9              17s 2s/step -
accuracy: 0.6628 - loss: 0.7821 - val_accuracy: 0.6885 - val_loss: 0.5465
Epoch 10/10
9/9              10s 1s/step -
accuracy: 0.7015 - loss: 0.7429 - val_accuracy: 0.6885 - val_loss: 0.5587

Training start time: 10:59:21
Training end time:   11:02:35
Total training duration: 194.20 seconds (3.24 minutes)
```

[28]:

```python
# Load the best checkpoint weights
vit_classifier.load_weights(checkpoint_filepath)

def plot_history(history, item):
    plt.plot(history.history[item], label=item)
    plt.plot(history.history["val_" + item], label="val_" + item)
    plt.xlabel("Epochs")
    plt.ylabel(item)
    plt.title("Train and Validation {} Over Epochs".format(item), fontsize=14)
    plt.legend()
    plt.grid()
    plt.show()

plot_history(history, "loss")
plot_history(history, "accuracy")
```


Train and Validation loss Over Epochs
loss
val_loss
7
6
5
4
3
2
1
loss
0
2
4
6
8
Epochs



1.5 Acknowledgment

I used a GPT-5.3-Codex to help scaffold and organize this notebook.

Gemini-3.1 was used to check the result and validate conformity with the assignment outline.

All observations and responses were written by me, Adam Nelson-Archer.